

KairoX ERP — Phase 2

The Procurement Engine: System, Business & Module Guide

Modules covered: Procurement (Stage 1) · BOM (Stage 2/3) · Inventory (Stage 4) · Supplier PO (Stage 5) · Intelligence (Chatbot & Forecasting)

Audience: this one document is written for three readers at once.

If you are...	Read	You will be able to
The client / business owner (non-technical)	Part A and Part B	Understand exactly what the software does, who touches it, and what happens at every step of a real order
A frontend developer	Part A, Part B, Part C	Design and build every screen without needing live data or a backend engineer
A backend developer	Part B, Part D, Appendices	Review the code with full context — you will know what each file is for before you open it

Companion document: *KairoX Phase-2 API Reference* — every endpoint with request/response examples and mock data.

Table of Contents

PART A — THE PLAIN-ENGLISH STORY

- 1. What Phase 2 is, in one page
- 2. The problem it solves
- 3. The five stages, told as a story
- 4. Who does what
- 5. What "done" looks like at each stage

PART B — THE BUSINESS FLOW IN DETAIL 6. Stage 1 — Intake: getting the paperwork in 7. Stage 2 — Breakdown & BOM generation 8. Stage 3 — The approval chain 9. Stage 4 — The inventory reality check 10. Stage 5 — Supplier purchase orders 11. The Production Board — the thread that ties it together 12. The Intelligence Service — the factory chatbot

PART C — FOR FRONTEND DEVELOPERS 13. The screen inventory 14. State machines you must render 15. Async patterns: 202-and-poll, SSE, and long jobs 16. The error contract 17. Badges, chips and status vocabulary

PART D — FOR BACKEND DEVELOPERS 18. Architecture and the layering rules 19. Module map — every file and what it is for 20. The data model 21. Cross-module choreography — who calls whom, and when 22. The key algorithms 23. Background work: Celery tasks and in-process sweepers 24. Configuration and migration notes 25. A code-review checklist

APPENDICES A. Glossary B. Complete enum reference C. Audit log actions

PART A — THE PLAIN-ENGLISH STORY

1. What Phase 2 is, in one page

Phase 1 of KairoX tracks garments **on the factory floor** — barcodes, cutting, stitching, attendance, wages. It assumes the materials are already there.

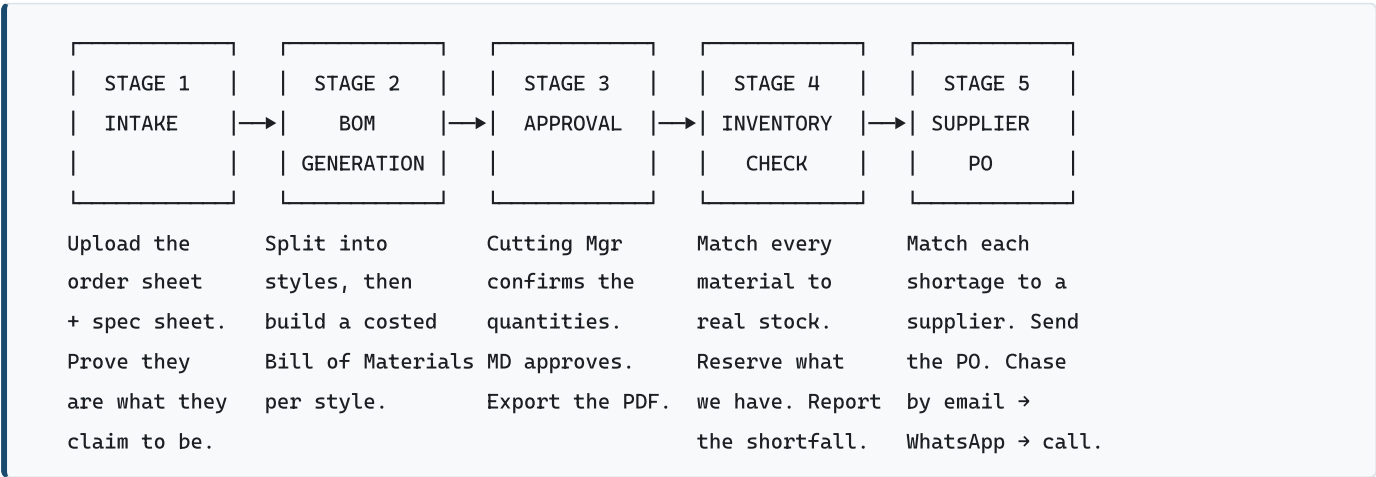
Phase 2 is about everything that has to happen before the floor can start. It is the answer to one question:

"A client just sent us an order. What material do we need, do we have it, and if not, who do we buy it from — and has it been confirmed?"

Today that question is answered by people, with spreadsheets, phone calls and memory. Phase 2 turns it into a system.

In one sentence: *Phase 2 reads the client's order and specification documents, works out the bill of materials automatically, checks it against real warehouse stock, and raises, sends and chases purchase orders with the suppliers who have historically sold us that exact article.*

The five stages



Everything above feeds **one board** — the Production Tracking board — which shows, per style, exactly how far along that style is on its journey from "order received" to "released to the factory floor".

2. The problem it solves

Before Phase 2

1. The client emails an order sheet and a spec sheet. Someone saves them in a folder.
2. The designer reads them and hand-builds a breakdown sheet in Excel.
3. The MD works out costing by hand — how much leather per jacket, times the order quantity, times the rate.
4. Someone walks to the warehouse or opens another spreadsheet to see whether the leather is in stock.
5. If it is short, someone remembers which supplier sold us that leather last time, calls them, and negotiates.
6. Someone types a purchase order in Word, emails it as a PDF, and then waits.
7. Three days later somebody asks *"did that supplier ever confirm?"* — and nobody knows.

Every one of those seven steps is a place where the order stalls, and no one can see that it has stalled.

After Phase 2

Problem	How Phase 2 fixes it
Wrong file uploaded to the wrong slot	Every upload is validated — the system checks the document really is an order sheet for this client, and rejects it with a specific reason if not
Consumption per garment is guesswork	The DCM cascade : the system remembers what was actually used last time this style was cut, and reuses it. Only if it has never seen the style does it estimate — and it flags the estimate
Nobody knows if stock is enough	The inventory check runs automatically the moment the MD approves, and returns a per-line verdict: sufficient / partial / out of stock, with the exact shortfall
Two orders quietly spend the same stock	The stock is reserved against the BOM in a soft ledger the moment it is checked. The second order sees it is already spoken for
"Which supplier do we use?"	The system reads the purchase ledger — who actually sold us this article, how recently, how often, at what rate — and ranks them
POs get typed by hand	POs are generated , grouped one per supplier, with GST computed correctly (CGST+SGST or IGST depending on the supplier's state)
Suppliers go silent	The escalation ladder : email → (5 hours) → WhatsApp → (5 hours) → automated phone call → hand back to the buyer. It stops the instant the supplier replies on <i>any</i> channel
Nobody knows where an order stands	The Production Board shows every style on a nine-rung ladder from "awaiting BOM" to "completed"

What Phase 2 deliberately does *not* do

- **It does not remove the human.** Every consequential step is a human decision the system prepares: the cutting manager confirms quantities, the MD approves, a manager cross-checks each PO. The system does the fetching, matching and arithmetic; people make the calls.
- **It does not guess silently.** When the system is unsure — an unrecognised material, two suppliers too close to call, an unconfirmed specification — it *stops and asks* rather than picking one. A wrong auto-selection gets rubber-stamped by a busy human; an explicit question does not.
- **It does not replace Phase 1's material module.** Phase 1's **material** module is the physical floor stock (leather lots, drawers, barcodes). Phase 2's **inventory** module is BOM-keyed procurement stock. They are bridged, not merged. (See §20.6.)

3. The five stages, told as a story

Let's follow one real order all the way through.

The order arrives

BOGGI, an Italian client, emails two files:

- **ORDER BOGGI MAIN SS27.xlsx** — the order sheet: which styles, which colours, which sizes, how many of each.
- **CLERMONT_SPEC.pdf** — the spec sheet: the measurements and construction detail for the CLERMONT jacket.

Stage 1 — Intake

The Direct Manager logs in and opens a **new submission** — an empty folder with two slots: one for the order sheet, one for the spec sheet.

They drag the order file into the order slot. The system:

1. Scans it for viruses.

2. **Sniffs it** — is this really an Excel file, or a `.pdf` renamed to `.xlsx`?

3. **Validates it** — does it look like a BOGGI order sheet? The system holds a per-client "template" that lists the anchor words and fingerprints it expects to find. If the cheap check is inconclusive, it escalates to an AI classifier.

If the file passes: **accepted**, the slot fills, and the system tells the DM whether the submission is now ready for Stage 2.

If it fails, the DM does not get a vague error. They get a **diagnostic**: *"This does not look like an order sheet. Expected to find: `ORDER CONFIRMATION`, `TAGLIA`, `QTY`. Found: `PACKING LIST`, `CARTON`. Closest matching profile: BOGGI packing list. Suggested fix: upload the order confirmation, not the packing list."*

Why this matters commercially: a wrong document caught here costs thirty seconds. The same wrong document caught at Stage 4 costs a week and a wrong purchase order.

The same happens for the spec sheet. Uploading the *same file twice* costs nothing — the system recognises the bytes by their fingerprint and replays the stored verdict without paying for a second AI call.

Stage 2 — Breakdown, then BOM

The order sheet holds **many styles**. So Stage 2 has two steps.

Step 1 — the breakdown. The DM asks the system to read the order document. Because reading a real order sheet with AI takes 30–200 seconds, this does not happen while the DM waits — the request returns immediately with *"queued"*, and a background worker does the work. The UI polls until it says *"ready"*.

What comes back is a **list of styles**, each with:

- its name and material,
- its total quantity and the quantity per size,
- its **colours**, each with their own per-size quantities,
- and — critically — a **suggested spec document** and a **suggested cutting pattern (DXF)**, matched by style code.

Those suggestions are marked `suggested`, never `confirmed`. **The system will not generate a BOM off a suggestion.** A human must click to confirm each one, or clear it and pick the right file. A plausible-but-wrong spec sheet produces a plausible-but-wrong BOM, which is far more dangerous than an empty screen.

Step 2 — generate the BOM. For each confirmed style, the DM triggers BOM generation (again queued in the background). The system:

1. Extracts the specification — measurements, materials, accessories.
2. Translates native measurement terms into standard codes. (An Italian spec says *"torace"*; a Japanese one says *"胸囲"*; both mean CHEST.)
3. Works out **DCM** — how many square decimetres of each material one garment consumes — through a strict cascade (see below).
4. Builds the line items: main material, lining, interlining, thread, accessories, manufacturing, packaging, FOB charges.
5. Prices each line and rolls it up into a per-garment FOB price and a bulk total.

The DCM cascade — the heart of Stage 2. No spec sheet ever states consumption. The system resolves it in strict order, and always stamps *where the number came from*:

Order	Source	Confidence	Meaning
1	Template — the consumption memory	0.95	We have cut this style before, and the cutting manager confirmed what it actually took. Best possible answer.
2	DXF — the cutting pattern file	0.88	We have the CAD pattern; measure the panels and apply the leather yield factor.
3	Similar style	0.70	We have not cut <i>this</i> style, but we have cut something close.
4	AI estimate	≤ 0.50	Last resort: a rough bounding-box from the finished measurements × wastage. Flagged for review.
—	Manual	1.00	A human typed it. Overrides everything.

The commercial payoff: the *first* order of a style may need an estimate. The second order of the same style hits the template and is exact — because when the cutting manager confirmed the first one, the system **wrote the confirmed number back into the memory**. The system gets more accurate the more you use it, without anyone maintaining a database.

Stage 3 — The approval chain

The BOM is a **draft**. Two gates stand between it and reality.

Gate 1 — the Cutting Manager. They open the BOM, see every line with its DCM and *where that number came from*, and correct anything wrong. When they click **Confirm Cutting**:

- The BOM moves to **ready_for_review**.
- **Every confirmed material quantity is written back into the consumption memory** — this is the learning step.
- The MD and DM each receive an in-app notification. If neither opens it within **two hours**, the system emails them.

Gate 2 — the Managing Director. They approve, reject with a reason, or approve-and-lock.

Approval is not a rubber stamp — it is the moment the order becomes real. On approve, three things happen automatically:

1. The **Client → Order → Style → SKU** hierarchy is created in the database. Until now, this order existed only as a parsed snapshot.
2. The **Production Board** advances the style to **BOM_APPROVED**.
3. **The inventory check runs immediately.**

The MD can also **export** the approved BOM as a PDF — the document that goes to the client for their approval.

If the MD **rejects**, the BOM freezes with the reason attached. The DM must explicitly **reopen** it, which bumps the revision and clears the cutting confirmation — so a rejected BOM cannot slip through without being re-confirmed.

A rule worth understanding: if anyone edits a *quantity* on a BOM that was already cutting-confirmed, the confirmation is automatically revoked and the BOM drops back to **draft**. You cannot change the numbers after the cutting manager signed off without the cutting manager signing off again.

Stage 4 — The inventory reality check

This runs automatically at approval, and can be re-run any time.

For every material line on the BOM, the system:

1. **Matches it to real stock.** Not by fuzzy guessing — in strict order: an exact normalised key match, then a curated alias (someone has previously confirmed that "SHEEP GLASS" means "SHEEP NAPPA GLAZED"), then a manual link. A fuzzy

near-match is only ever offered as a **suggestion**, never applied automatically.

2. **Sums the stock across lots**, converting units where needed (a stock lot in metres against a BOM line in square decimetres). If the units cannot be reconciled, the line is flagged and treated conservatively as out of stock — the system would rather send you to buy material you already have than let you start cutting material you don't.
3. **Subtracts what other BOMs have already reserved.** `available = on hand - reserved by everyone else`.
4. **Reserves** what this BOM can claim: `min(required, available)`.
5. **Computes the shortfall** and stamps a verdict: `sufficient`, `partial` or `out_of_stock`.

The result is a report with a summary badge (the worst line wins), a per-line grid, and a total shortfall value in rupees.

Two design points that matter. First, **reservations never touch the on-hand number**. They are a separate ledger, so a fresh stock upload can replace the whole master without destroying a commitment. Second, the candidate stock rows are **locked for the duration of the check**, so two approvals happening at the same second cannot both reserve the same leather.

Lines the check **ignores**: `manufacturing` and `fob_charge`. Those are services and charges — there is no such thing as having them in stock.

Stage 5 — Supplier purchase orders

Now the system knows exactly what is short. For each shortfall line it asks: *who sells us this?*

Supplier matching is deterministic-first, and evidence-based:

Order	Method	What it means
1	<code>ledger</code>	The purchase history shows this exact article bought from this supplier before.
2	<code>alias</code>	Rewrite the term through a curated alias and try the ledger again.
3	<code>category</code>	No article match, but this supplier is our usual vendor for this <i>kind</i> of material. Ranked lower — this is a guess.
4	<code>suggestion</code>	A fuzzy near-match, offered only . Never auto-selected.
5	<code>unresolved</code>	Nothing confident. The system holds a <code>needs_supplier</code> draft PO and asks the buyer.

Suppliers are ranked on **recency × frequency × contactability – rate**. Recency dominates — who bought it most recently is the strongest signal. Contactability is a *ranking term*, not a filter: a frequent supplier with no email still ranks top, but their PO is flagged `no_contact_channel` so the buyer knows to phone them.

If the top two suppliers are too close to call, the system does not choose. It marks the PO `ambiguous` and shows the buyer the shortlist with the evidence — last rate, last purchase date, transaction count.

Then:

- **Group** — all lines for the same supplier become **one PO**, not one PO per line.
- **Price** — GST is computed from the supplier's state code against ours. Same state → CGST + SGST. Different state → IGST.
- **Cross-check** — the PO is submitted for approval, routed by material type: **leather POs go to the Cutting Manager**; accessory and service POs go to MD/DM/HR. The MD can approve anything.
- **Send** — on send, the system allocates the financial-year PO number (`PO-07(25-26)`), renders the PDF, emails it to the supplier with the PDF attached, and starts the clock.
- **Track** — the email carries an invisible tracking pixel and wrapped links. The system knows when the supplier *opened* it and when they *clicked*. This is self-hosted; no third-party email SaaS sees your supplier list.

- **Escalate** — if there is no acknowledgement in five hours: WhatsApp. Five hours more: an automated phone call. Five more: give up and notify the buyer to follow up personally.
- **Stop** — the ladder halts the instant the supplier acknowledges **by any channel** — a reply on WhatsApp, pressing 1 on the call, or a manager marking it confirmed by hand.

A hard email bounce short-circuits the whole ladder straight to WhatsApp. There is no point spending five hours waiting on a dead address.

The board

Through all of this, one row per style advances up a nine-rung ladder:

```
awaiting_bom → bom_approved → inventory_checked → po_raised → po_confirmed
→ material_ready → released_to_production → in_production → completed
```

Most rungs are **automatic** — a side-effect of the stage that just completed. Two are **human**: **released_to_production** is the deliberate go/no-go, and any rung can be manually corrected by the MD, DM or Cutting Manager if reality and the system disagree.

material_ready is reached only when **every** PO for that style is confirmed.

4. Who does what

Role	In Phase 2 they...
Managing Director (managing_director)	Superuser. The sole approver, rejecter and exporter of a BOM. Can approve any purchase order. Deactivates and reactivates suppliers.
Direct Manager (direct_manager)	The operator of the pipeline. Uploads documents, runs the breakdown, confirms spec/DXF matches, triggers BOM generation, runs inventory checks and syncs, generates POs, edits them, sends them, records manual acknowledgements.
Cutting Manager (cutting_manager)	Reads and edits the BOM; confirms cutting — the gate that unlocks MD approval and teaches the consumption memory. Approves leather purchase orders.
HR (hr)	Can approve accessory and service purchase orders (the cross-check pool).
Viewer (viewer)	Read-only across inventory, suppliers, POs and the production board.
Suppliers (no login)	Never log in. They receive an email with a PDF, reply on WhatsApp, or press 1 on a call. Their responses reach the system through unauthenticated, token-bearing webhook URLs.

Everything in Phase 2 requires a login. Unlike Phase 1's shop floor — where workers are identified by barcode and have no account — Phase 2 is entirely a management surface.

Two routes are deliberately open (no login): the email tracking pixel and click-redirect (**/t/o/...** , **/t/c/...**) and the provider webhooks (**/webhooks/ses** , **/webhooks/twilio/***). A supplier's mail client and Twilio's servers call these; they carry an opaque per-send token, not a session.

5. What "done" looks like at each stage

A practical checklist. If you are wondering "can we move on?", this is the answer.

Stage	Done when	Where you see it
1 — Intake	Both slots hold an accepted document with a clean (or skipped) virus scan	<code>ready_for_stage_2: true</code> on the submission
2a — Breakdown	The style list has been extracted and persisted	Breakdown status is <code>ready</code>
2b — Attachments	Every style's spec is <code>confirmed</code> or <code>none</code> — never left at <code>suggested</code>	Each style's <code>spec_match_status</code> chip
2c — BOM	Each style has a <code>bom_id</code>	The style row shows its BOM link
3a — Cutting	<code>cutting_confirmed_at</code> is set; consumption templates written back	BOM status <code>ready_for_review</code>
3b — Approval	MD approved; the order/style hierarchy now exists	BOM status <code>approved</code> or <code>locked</code>
4 — Inventory	A <code>complete</code> check exists with a per-line verdict and reservations held	Check badge; board at <code>inventory_checked</code>
5a — PO generation	Every shortfall line sits on a PO; nothing is <code>needs_supplier</code>	<code>needs_supplier: 0</code> in the generate response
5b — Cross-check	Every PO is <code>approved</code>	PO status
5c — Sent	Every PO has a <code>po_number</code> , a PDF, and <code>sent_at</code>	PO status <code>sent</code> ; board at <code>po_raised</code>
5d — Confirmed	Every PO has <code>acknowledged_at</code>	PO status <code>confirmed</code> ; board at <code>po_confirmed</code> → <code>material_ready</code>
Handover	A human presses go	Board at <code>released_to_production</code> — Phase 1 takes over

PART B — THE BUSINESS FLOW IN DETAIL

6. Stage 1 — Intake: getting the paperwork in

Module: `app/modules/procurement/` · URL prefix: `/api/v1/procurement`

6.1 The submission — why a "batch" exists

An order sheet and a spec sheet are two files that describe **one job**. They arrive separately, possibly hours apart, possibly from different people. The system needs something to pair them.

That is the **submission**: a small record with two slots.

```
Submission
├─ order_document_id  — the CURRENT order sheet
├─ spec_document_id   — the CURRENT spec sheet
├─ client_id          — may be unknown at upload; derived later
├─ client_order_id    — filled in at MD approval, when the order becomes real
└─ status             — open → complete → consumed | rejected
```

Two subtleties worth knowing:

- **The slots are pointers, not the archive.** Every document ever uploaded into this submission — including superseded ones — is kept, linked by `Document.submission_id`. The slot just says "*the current order sheet is this one*". Re-uploading a corrected file supersedes the old one; the old one is never deleted.
- **You do not have to open a submission first.** Two convenience endpoints (`/upload/order-sheet` , `/upload/spec-sheet`) create the submission on the fly — but **only if the file passes validation**. A rejected file returns diagnostics and no submission is created. You then use the returned `submission_id` for the paired upload.

6.2 The validation pipeline

Every upload runs the same five-step gauntlet. All of it runs **off the request thread** (in a threadpool) because Excel parsing, PDF parsing and AI calls are blocking.

```

bytes in
|
▼
① SIZE GATE ————— over MAX_UPLOAD_MB? → 413, streamed abort
|
▼
② VIRUS SCAN ————— infected? → 422 + audit row
|                          scanner down? → 503
|
▼
③ MIME SNIFF ————— libmagic: is it really what the extension claims?
|                          unsupported? → 415
|
▼
④ CLASSIFY ————— (a) heuristic: anchors + fingerprints from the client template
|                          (b) if inconclusive → LLM classifier
|                          (c) if a scanned PDF → skip the text layer, go straight to vision
|                          wrong kind? → 422 with full diagnostics
|
▼
⑤ PERSIST ————— Document row (accepted OR rejected – both are cached by sha256)
|                          accepted → fill the slot, supersede the old one, recompute the gate

```

6.3 Client templates — how onboarding a new client works

The validator does not have BOGGI's rules hard-coded. It reads a `client_template` row: a per-client × per-document-kind profile holding the anchor words, fingerprints, accepted MIME types, expected layout, language, size system, currency and confidence thresholds.

Onboarding a new client is one registry row, not a code change. Templates are seeded idempotently from `config/client_templates.yaml`.

`expected_layout: scanned_pdf` is a particularly useful hint: it tells the validator not to waste time on a text-layer pass that will find nothing, and escalate straight to the vision classifier.

6.4 Idempotency — uploading the same file twice

Every document is fingerprinted with `sha256`, and the hash is globally unique.

- **Same file, same slot, same submission** → the stored verdict is replayed. No re-validation, **no second AI bill**. If it was accepted, the slot is re-pointed; if it was rejected, the same diagnostics come back.
- **Same bytes, different submission — or the other slot of this one** → `409 duplicate_content`, with an explanation of where those bytes already live and what to do about it. Reusing them would either skip validation or dead-end the slot.

6.5 The `needs_manual_review` verdict and the `force` override

Sometimes the classifier is neither confident-yes nor confident-no. That produces `needs_manual_review` — a soft rejection.

A DM or MD can retry with `?force=true`, which means *"I have looked at this file and I vouch for it."* The document is accepted and the pipeline proceeds.

Hard rejections are never overridable. A virus, an unsupported MIME type, an oversized file or an empty/corrupt file cannot be forced through.

6.6 The readiness gate

`ready_for_stage_2` is true when **both** slots hold a document whose `validation_status` is `accepted` **and** whose `scan_status` is `clean` or `skipped` (`skipped` means scanning is disabled in development).

The response also carries a `blocking` array in plain language — `["spec_sheet_missing"]`, `["order_sheet_needs_manual_review"]` — which the UI can render directly.

6.7 The lock

Once Stage 2 claims a submission, its status flips to `consumed` and **the submission is locked**. Further uploads are refused. The claim is an atomic compare-and-set from `complete` to `consumed`, so two people clicking "generate breakdown" at the same moment cannot both enqueue the job — exactly one wins.

7. Stage 2 — Breakdown & BOM generation

Module: `app/modules/bom/` · **URL prefix:** `/api/v1/procurement`

7.1 Why breakdown and BOM are two separate steps

An order sheet contains many styles. A BOM is **per style**. Earlier designs tried to do both in one action and it did not work, for two reasons:

- 1. **Each style needs its own spec sheet and its own cutting pattern.** The order sheet does not say which spec file belongs to which style — a human has to confirm that.
 - 2. **Fan-out.** Ten styles means ten independent AI extractions. If one fails, the other nine should still work.
- So: **breakdown first** (one job over the order document, produces the style list), **then BOM per style** (one job each).

7.2 The breakdown

`POST /submissions/{id}/order-breakdown` returns `202 Accepted` immediately with a Celery task id. Extraction takes 30–200 seconds; it never runs on the request path.

The endpoint is **idempotent in three directions**:

State	Response	Meaning
Rows already exist	<code>already_ready</code>	Nothing re-runs
Claim held, rows not written	<code>already_processing</code>	A worker is mid-flight; no double-enqueue
Nothing yet, claim succeeds	<code>queued</code> + <code>task_id</code>	The job just started
Claim fails	<code>409</code>	Order document not accepted, or someone else claimed it

The UI then polls `GET /submissions/{id}/order-breakdown`, which returns `not_started`, `processing`, or `ready` with the style list.

What one style looks like:

OrderStyle

— style_signature	the cross-order-stable key (e.g. "CLERMONT")
— style_name	as written on the order sheet
— material	as written on the order sheet
— qty	total units
— per_size_qty	{"46": 10, "48": 20, "50": 20, "52": 10}
— warnings	anything the extractor was unsure about
— spec_document_id	+ spec_match_status: none suggested confirmed
— pattern_reference_id	+ dxf_match_status: none suggested confirmed
— bom_id	null until generated
— colors[]	each with its own color_key, label, qty, per_size_qty

7.3 Suggestion, never silent binding

When the breakdown is built, the system tries to pair each style with a spec document and a DXF pattern already on file for that client. The matching is careful:

1. **Style code first.** Extract alphanumeric codes from the style name and the candidate filename, normalising separators — `CLERMONT_15-06-26-P53` yields `P53`; `SP-64806 (SP74006)` yields `SP64806` and `SP74006`. This survives real CAD filenames.
2. **Exactly one hit → suggest it.** Two or more hits → **suggest nothing**.
3. **Fallback:** shared normalised words of more than two characters. Again, exactly one hit or nothing.

The reasoning: *"a wrong pre-selection in the UI gets rubber-stamped"*. An ambiguous match that forces a human click is safer than a confident-looking wrong one.

The DM then calls `POST /order-styles/{id}/attachments` to confirm or override:

Body	Effect
<code>{}</code>	Accept the current suggestions — they become <code>confirmed</code>
<code>{"spec_document_id": "..."} </code>	Override with this specific document — <code>confirmed</code>
<code>{"clear_spec": true}</code>	Detach — status becomes <code>none</code>
Both an id and its <code>clear_*</code> flag	<code>422</code> — a contradiction

7.4 Generating a BOM

`POST /order-styles/{id}/generate-bom` — again `202` + a task id. **Preconditions are checked before queuing**, so the operator sees a `4xx` immediately rather than a task failure later:

- Style does not exist → `404`
- Already has a `bom_id` → `already_generated` (idempotent replay)
- Spec is still `suggested` → `422` — **confirm or clear it first**

That last one is the important rule. **A style may be generated with no spec at all** (`spec_match_status: none` → the BOM carries a `spec_pending` warning). What it may *not* be generated with is a spec nobody looked at.

7.5 Inside BOM generation

1. EXTRACT SPEC	→ ExtractedSpec (measurements, materials, accessories, warnings) staged raw into spec_extraction (the audit truth)
2. RESOLVE POMs	→ native terms → standard codes seeded dictionary → alias keywords → fuzzy → constrained LLM written to pom_measurement, one row per size per code
3. FLATTEN ATTRIBUTES	→ the spec's usable facts
4. RESOLVE PATTERN	→ "follow pattern X in size Y" → a real DXF / template
5. BUILD LINE SEEDS	→ one seed per BOM line, from the spec + the cost catalogue
6. RESOLVE DCM	→ the four-source cascade, per material line
7. COST	→ per line and rolled up to the header
8. PERSIST	→ bom + bom_item rows

Steps 1–8 are **ONE transaction**. Rows are staged with `add()` / `flush()` and committed exactly once at the end. A failure halfway leaves nothing half-written.

7.6 The nine line categories

Category	DCM-resolved?	In the inventory check?	Notes
main_material	✓	✓	The primary leather
sub_material	✓	✓	Secondary leather
lining	✓	✓	
interlining	✓	✓	
thread	✗	✓	Given quantity and price
accessory	✗	✓	Buttons, zips. Buyer-supplied ones carry price 0
packaging	✗	✓	
manufacturing	✗	✗	A service — no stock exists
fob_charge	✗	✗	A charge — no stock exists

7.7 The costing arithmetic

Deliberately simple, entirely in `Decimal`, and computed **at write time only** — never on read, never trusting a number from the client.

PER LINE		
total_cost = qty_per_garment × unit_price		the per-garment cost of this line
bulk_qty = order_qty × qty_per_garment		e.g. 60 × 34.5 = 2,070 dm²
line_bulk = order_qty × total_cost		e.g. 60 × 62.10 = \$3,726
PER BOM		
garment_fob_price = ∑ total_cost		the FOB price of one garment
bulk_total = ∑ line_bulk		= order_qty × garment_fob_price

Money quantises to two decimal places, quantities to three. A material line's `qty_per_garment` is its DCM.

7.8 The editable contract

`PATCH /boms/{id}/items` is a **bulk** edit under an **optimistic revision lock**.

You send the `base_revision` you were looking at and a list of `{bom_item_id, field, value}` edits, where `field` is one of `dcm`, `qty_per_garment` or `unit_price`.

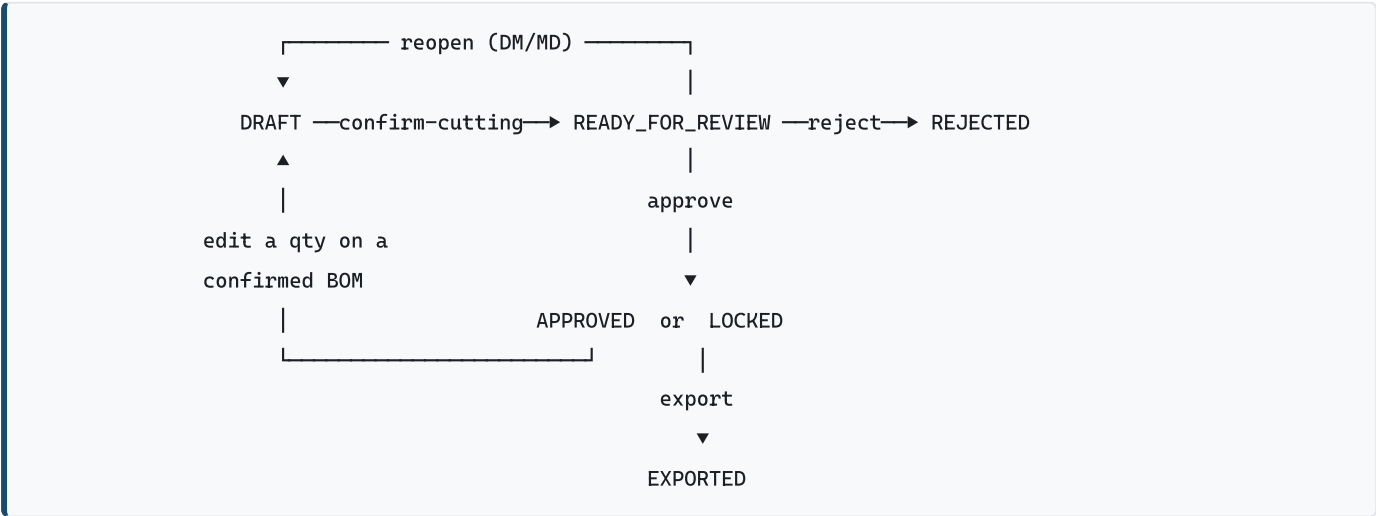
The server:

1. Refuses if the BOM is not editable (`draft` or `ready_for_review` only). A rejected BOM says "reopen it first"; an approved one says `bom_locked`.
2. Refuses on a stale revision — `409` with the current revision, so the UI can reload and re-present.
3. Applies the edits, validating each (unknown item, unsupported field, non-numeric, negative → `422`).
4. **Stamps any quantity edit on a material line as `dcm_source = manual` , **confidence 1.00**.** A human number is the truth.
5. **If a quantity changed and the BOM was already cutting-confirmed, it revokes the confirmation** and drops the BOM to `draft` , returning `reconfirm_required: true` .
6. Recomputes the whole tree and returns it, along with the new revision.

The revision claim is an atomic compare-and-set; if it loses the race, the transaction rolls back and you get `409` with the fresh revision.

8. Stage 3 — The approval chain

8.1 The BOM state machine



Status	Editable?	What it means
<code>draft</code>	✓	Freshly generated, or reopened, or its confirmation was revoked
<code>ready_for_review</code>	✓	The cutting manager has confirmed; awaiting the MD
<code>approved</code>	✗	MD approved. Order hierarchy created. Inventory check has run
<code>locked</code>	✗	Approved with <code>lock: true</code> — the same, said louder
<code>rejected</code>	✗	Frozen with a reason. Must be reopened to edit
<code>exported</code>	✗	A PDF has been rendered and stored

8.2 Confirm cutting — the learning step

`POST /boms/{id}/confirm-cutting` (Cutting Manager; MD/DM bypass) does four things:

1. Stamps `cutting_confirmed_by` and `cutting_confirmed_at`; status → `ready_for_review`.
2. **Writes every material line's confirmed quantity into `style_consumption_template`**, keyed on the cross-order-stable style signature. *This is why the second order of a style is exact.*
3. If a DXF pattern exists, records a **yield observation** — the ratio between the pattern's net area and the confirmed consumption. The system learns the real leather yield per species.
4. Creates review notifications for the MD and DM, each with its own two-hour escalation clock.

The response tells you how many templates were back-filled and how many notifications were created.

The style signature — the key the memory is stored under — is resolved as: buyer's `customer_ref` → internal `internal_ref` → a slug of the style name, uppercased. It deliberately does **not** use `style_id`, because style rows are order-scoped and a new order creates a new style row. Keying on the signature is what makes the memory work across orders.

8.3 Approve

`POST /boms/{id}/approve` (MD only). Optional body `{"lock": true}`.

It refuses outright if `cutting_confirmed_at` is null — `409 cutting_confirmation_required`. The MD cannot approve a BOM the cutting manager has not seen.

On success:

1. **Materialise the breakdown** — create the Client → Order → Style → SKU tree from the parsed order snapshot, and link the BOM to it. Idempotent: a BOM that already has a style is left alone. If the order sheet never resolved a client, this is skipped with a warning (the BOM is still approved).
2. Set status, `approved_by`, `approved_at`, `locked_at`.
3. Write the `BOM_APPROVE` audit row.
4. **Advance the production board** to `bom_approved`.
5. **Run the inventory check** and return its id.

Steps 4 and 5 are best-effort — they are wrapped so that a failure in the board or the check does not fail the approval. The BOM is approved; the check can be re-run.

8.4 Reject, reopen, export

Reject (MD, mandatory reason) — only from `ready_for_review`. An approved BOM cannot be rejected; cancel and re-issue instead.

Reopen (DM or MD) — only from `rejected`. Bumps the revision, clears the rejection **and clears the cutting confirmation**. The chain restarts properly.

Export (MD) — only from `approved` / `locked` / `exported`. Renders a PDF (WeasyPrint → ReportLab → raw HTML, in that fallback order, so an export *always* produces a durable artifact), deduplicates it by sha256, stores it, and links it to the BOM. Re-exporting an already-exported BOM returns the existing document rather than making a second one.

8.5 The notification system

Notifications are **rows in a table**, not messages on a wire. Delivery is a view over those rows. That means:

- `GET /notifications` polls them.
- `GET /notifications/stream` pushes them over Server-Sent Events, emitting the unseen backlog on connect and then polling every ten seconds with keep-alive comments.

- **Both read the same rows.** A dropped SSE connection loses nothing.

`POST /notifications/{id}/open` stamps `opened_at`. **That is the signal that cancels the email escalation.** If nobody opens the notice within `bom_review_escalation_hours` (default 2), an in-process sweeper emails the recipient. The sweep is DB-driven and idempotent — it survives restarts and never double-emails.

9. Stage 4 — The inventory reality check

Module: `app/modules/inventory/` · **URL prefix:** `/api/v1/procurement`

9.1 The inventory master

Stock lives in `inventory_item`: a normalised, deduplicated master with `description`, `normalized_key`, `uom`, `qty_on_hand`, `rate`, `category`, `colour`, `article_ref`, `is_active`.

It is kept current by uploading a spreadsheet:

- `POST /inventory/preview` — a dry run. Parses and normalises, writes nothing, and reports what would be kept, dropped, merged, and any warnings. **Always show this before committing.**
- `POST /inventory/commit` — the same parse, then an idempotent upsert keyed on `normalized_key`. **The sheet wins on quantity.** Keys absent from the sheet are **soft-deactivated**, never deleted.

9.2 The three numbers

<code>qty_on_hand</code>	a real column – what the warehouse says is there
<code>reserved</code>	Σ of ACTIVE reservations – a separate ledger
<code>available</code>	<code>qty_on_hand</code> – <code>reserved</code> ← DERIVED, never stored

`available` is never stored, so it can never drift from its inputs. This mirrors Phase 1's material module exactly.

Reservations never mutate `qty_on_hand`. This is what lets a fresh sheet snapshot-replace the whole master without destroying a live commitment.

A reservation has three states: `active` (counts against available), `released` (freed), `consumed` (physically issued — terminal).

9.3 Running the check

`POST /boms/{id}/inventory-check`, and automatically at MD approval.

1. GUARD	BOM must be approved / locked / exported, else 409
2. RELEASE	drop this BOM's prior reservations (reason: "recheck")
3. PARTITION	stockable lines vs excluded (manufacturing, fob_charge)
4. BUILD KEYS	one normalised key per line, plus alias targets and a longest-token probe – enough to fetch a candidate pool without loading the whole master
5. FETCH + LOCK	SELECT ... FOR UPDATE on the candidates
6. SUM RESERVATIONS	what other BOMs already hold against those rows
7. HEADER	write the InventoryCheck row (status: running)
8. PER LINE	match → convert → compute → write the line → reserve across lots
9. COMPLETE	status → complete, audit row, advance the board

Everything is one transaction, with the candidates locked. Two approvals in the same second cannot double-spend the same leather.

9.4 Per-line matching

```
exact normalized key → found?  method = key
  | no
  ▼
curated alias rewrite → found?  method = alias
  | no
  ▼
fuzzy token overlap → above the floor? → SUGGESTION ONLY
  |                                     flags: ["suggestion", "unmatched"]
  ▼
nothing → flags: ["unmatched"]
```

A `manual` method exists for a human-confirmed link. **A fuzzy candidate is never auto-bound.** Confirming a suggestion writes a `material_alias` row — so the system learns, and next time it is a clean `alias` match.

9.5 Units

Stock lots and BOM lines do not always share a unit. `uom_conversion` holds `from_uom → to_uom → factor` pairs (identity rows included), seeded from `config/uom_conversions.yaml`.

If a lot's unit cannot be converted into the BOM line's unit, that lot contributes **nothing** and the line is flagged `uom_mismatch`. If *no* lot converts, the line is conservatively `out_of_stock`.

Why conservative: telling you to buy material you already have wastes money. Telling you that you have material you cannot actually use stops the line. The second is worse.

9.6 The per-line arithmetic

```
required      = bom_item.bulk_qty          (order_qty × qty_per_garment, from Stage 2)
on_hand       = Σ over matched lots of convert(lot.qty_on_hand → bom line uom)
reserved_by_others = Σ active reservations on those lots, excluding this BOM
available     = max(0, on_hand - reserved_by_others)
reserve       = min(required, available)    ← what this BOM claims
shortfall     = max(0, required - reserve)   ← what Stage 5 must buy

status = sufficient  if available ≥ required > 0 and units reconciled
      = partial      if available > 0
      = out_of_stock otherwise
```

The reserve is then spread across the matched lots, largest first, capped per lot by what is still free on that lot.

9.7 Reading the results

- `GET /inventory-checks/{id}` and `GET /boms/{id}/inventory-check` re-render a stored check **from its persisted rows**. There is no recompute on read — the stored rows *are* the truth.
- `GET /inventory-checks` is the grouped dashboard: **client** → **order** → **style**, each style carrying its badge and shortfall-line count, plus totals.

The **badge** is the worst line status in the check: `out_of_stock` beats `partial` beats `sufficient`. The summary also carries a `shortfall_value` — the shortfall quantity times the stock rate, in rupees.

10. Stage 5 — Supplier purchase orders

Module: `app/modules/supplier_po/` · **URL prefix:** `/api/v1/procurement`

10.1 The supplier directory

Suppliers can be created one at a time or imported in bulk from the same spreadsheet the factory already keeps.

The import is a two-step **preview** → **commit**, and it does three things:

1. **Upsert suppliers.** Existing suppliers are enriched, **never blanked** — `existing.phone` or `new.phone`. A supplier's known email is not wiped by a sheet that omits it.
2. **Rebuild the supply history wholesale.** `supplier_supply_history` is a pre-aggregated (*supplier, article*) ledger — how many transactions, first and last purchase dates, last/min/max rate — normalised with the **same normaliser Stage 4 uses**, so the two modules agree on what an article is called.
3. **Soft-deactivate** suppliers absent from the import. Never hard-delete.

A supplier's **type** (`leather` / `accessory` / `service`) defaults from the dominant mode in their purchase history. It drives two things: which PO template renders, and **who has to approve their POs**.

`supplier.email_status` (`unknown` / `valid` / `invalid`) is set by bounce feedback and gates whether email is attempted at all.

10.2 Generating POs

`POST /boms/{id}/generate-pos`.

- Refuses if no inventory check has run — `409 no_inventory_check`.
- If non-cancelled POs already exist for the BOM, returns them with `already_generated: true`. **Idempotent**.
- Takes the check's lines where `status != sufficient` and `shortfall_qty > 0`.
- Matches each to a supplier, groups by supplier, and creates one PO per supplier.
- Unmatched lines each become their own **needs_supplier draft** — a held PO with the candidate shortlist attached, waiting for a buyer to assign the vendor.

Each PO line pre-fills its unit price from the **supplier's last rate** for that article (falling back to the BOM's unit price) and its unit from the supplier's historical unit. Every line keeps its `bom_item_id` and `inventory_item_id` back-links, so you can always trace a PO line back to the shortfall that caused it.

10.3 Supplier ranking

```
score = W_RECENCY    × recency(last_purchase)    ← decays ~half per year
      + W_FREQUENCY  × frequency(txn_count)
      + W_CONTACT    × has_contact
      - W_RATE        × normalised_rate           ← tie-break only
```

Two rules govern whether the system chooses at all:

- A **category-only** hit is *weak* — it is a guess about the kind of material, not evidence about this article. Held as ambiguous.
- If the top two scores are within the tie band, it is **too close to call**. Held as ambiguous.

In both cases the buyer sees the shortlist with the evidence: score, transaction count, last rate, last purchase date, and whether the supplier is contactable.

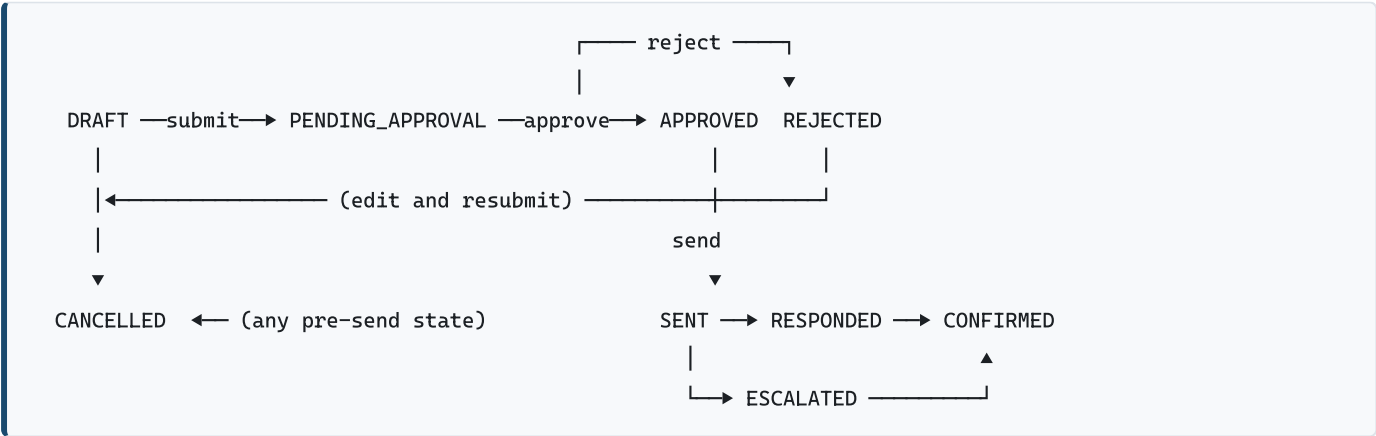
10.4 GST

Computed from the buyer's state code (configured; Tamil Nadu = 33) against the supplier's state code (derived from their GSTIN):

Condition	Mode	Tax
Same state	INTRA	CGST + SGST, half the rate each
Different state	INTER	IGST at the full rate

Default rate is 12% — so 6 + 6 intra, or 12 inter. The PO carries subtotal , cgst , sgst , igst , gst_mode , round_off and total .

10.5 The PO state machine



Status	Meaning
draft	Generated or reopened. Fully editable
pending_approval	Submitted for cross-check
approved	Cross-checked. Ready to send
rejected	Sent back with a reason. Returns to draft on edit
sent	PO number allocated, PDF rendered, dispatched. Frozen
responded	The supplier engaged (opened / clicked)
escalated	No acknowledgement; the ladder is climbing
confirmed	Acknowledged. The ladder stops
cancelled	Terminal. Reachable from any pre-send state

A sent PO is frozen. You cannot edit it; you cancel and re-issue. The supplier has a PDF in their hand — changing our copy behind their back is not an option.

10.6 Editing a PO

PATCH /pos/{id}/items uses exactly the same contract as the BOM: base_revision plus a bundle of changes — item edits, header edits, added items, removed items. Same 409 stale_revision . Additionally 409 po_locked once sent.

Changing the supplier re-routes the approver and resets any approval. You cannot approve a PO for supplier A and then quietly swap in supplier B.

10.7 The cross-check

`POST /pos/{id}/submit` requires a supplier assigned and at least one line. It notifies the routed approvers in-app.

Routing by material type:

Supplier type	Who may approve
leather	Cutting Manager, MD
accessory , service , or unset	MD, DM, HR
anything	MD (superuser)

A wrong-role approval attempt gets `403 wrong_approver` naming the roles that can, so the UI can tell the user who to ask.

10.8 Sending

`POST /pos/{id}/send` — only from `approved`. In order:

1. **Allocate the PO number** — financial-year scoped, April to March. `2025-04` → `25-26`.
2. **Set the issue date** and mint an opaque tracking token.
3. **Render the PDF** using the template for this supplier's type. Deduplicated by sha256 and stored, exactly like the BOM export.
4. **Route by contact:**
 - Valid email → send it, with the PDF attached, the tracking pixel injected and links wrapped. `current_rung = 0`.
 - No email but a phone → skip email; the sweeper's first rung will WhatsApp them.
 - Neither → flag `no_contact_channel: true`. A human must handle it.
5. **Start the clock** — `next_escalation_at = now + po_escalation_hours` (default 5).
6. **Advance the board** to `po_raised`.

10.9 Tracking

Self-hosted, no third-party email SaaS:

- `GET /t/o/{token}.gif` — a 1×1 pixel. Fetching it stamps `first_opened_at` and moves the latest response from `pending` to `opened`.
- `GET /t/c/{token}?u=...&s=...` — a click redirect. The target URL is **signature-verified** before redirecting, so the link cannot be rewritten into an open redirect. Stamps `first_clicked_at`, then `302` s to the real destination.
- `POST /webhooks/ses` — delivery, bounce and complaint events. **A hard bounce marks the supplier's email invalid, fails the response, and sets `next_escalation_at` to now** — escalate on the very next sweep rather than burning five hours on a dead address.

Every event lands in one unified `po_tracking_event` log.

10.10 The escalation ladder

Driven by an in-process sweeper. **One rung per due PO per sweep**, idempotent.

Rung	After	Action	On failure
0 → 1	5h from send	WhatsApp to <code>whatsapp_phone</code> or <code>phone</code> . Status → <code>escalated</code>	Logged as <code>no_whatsapp</code> ; the rung still advances
1 → 2	5h more	Automated call — text-to-speech reads the PO reference	Logged as <code>no_phone</code> ; the rung still advances
2 → 3	5h more	Give up. In-app notification: "no response after email, WhatsApp and a call — follow up manually." Clock cleared	—

The ladder stops the moment the supplier acknowledges by any channel:

Channel	How
WhatsApp reply	<code>POST /webhooks/twilio/whatsapp</code> with the tracking token
Phone — press 1	<code>POST /webhooks/twilio/voice</code> with <code>Digits: "1"</code>
Manual	<code>POST /pos/{id}/acknowledge</code> — a DM/MD records it, optionally with the confirmed quantity

Acknowledgement is **idempotent** — a second acknowledgement is a no-op. It sets `acknowledged_at` , `acknowledged_channel` , status `confirmed` , clears `next_escalation_at` , and advances the board.

The transport is pluggable: `log` (prints to stdout — the whole ladder is testable offline with no Twilio account), `noop` , or `twilio` (WhatsApp + Voice through one vendor).

11. The Production Board — the thread that ties it together

One row per (order, style). Nine rungs.

```
awaiting_bom → bom_approved → inventory_checked → po_raised → po_confirmed
              → material_ready → released_to_production → in_production → completed
```

Rung	Driven by	Trigger
<code>awaiting_bom</code>	—	The row is created
<code>bom_approved</code>	System	<code>BomService.approve_bom</code>
<code>inventory_checked</code>	System	<code>InventoryService.run_check</code>
<code>po_raised</code>	System	<code>PoService.send_po</code> — the first PO sent
<code>po_confirmed</code>	System	<code>PoService._acknowledge</code>
<code>material_ready</code>	System	When every PO for the style is confirmed
<code>released_to_production</code>	Human	The deliberate go/no-go — MD, DM or Cutting Manager
<code>in_production</code>	Human	
<code>completed</code>	Human	

The board only moves forward. A system edge never moves backwards. The manual transition endpoint *can* force a correction — that is deliberate, for when reality and the system disagree — and every transition is audited.

Every system edge is best-effort. If advancing the board fails, the stage that triggered it still succeeds. The board is a view of progress, not a lock on it.

`GET /production-tracking` returns the board grouped by client and order, filterable by either.

12. The Intelligence Service — the factory chatbot

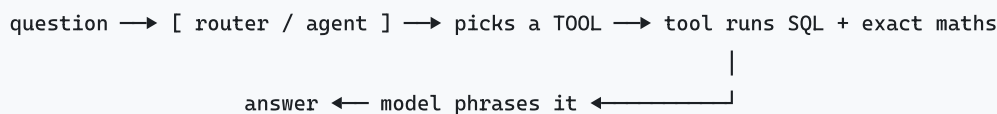
Module: `app/modules/intelligence/` · URL prefix: `/api/v1/chat`

12.1 What it is for

A natural-language window onto the factory: "Is CLERMONT on schedule?", "Where is the bottleneck?", "What should we run next week?"

12.2 Why the maths is not in the model

The useful questions are arithmetic over SQL — how many ordered, how many produced, what rate is needed to hit the deadline. So the architecture puts the maths in **tools** and leaves the model to route and phrase:



The model never does arithmetic. That makes answers **exact** and lets a small, cheap, local model do the job.

12.3 Two interchangeable backends, one response shape

Backend	When	Cost
Deterministic router (default)	Always available. Keyword and entity routing over the same tools	Zero — no model
LangGraph ReAct agent	When <code>CHAT_MODEL</code> is configured. Reasons over multi-step questions	Model cost

Both return `{answer, tool, data}`. The frontend never changes, and never breaks — if no model is configured, `use_llm: true` silently falls back to the deterministic router rather than erroring.

12.4 The tools

Tool	Answers
<code>tool_schedule_status</code>	Is this style on track? Ordered vs produced, current rate, required rate, deadline
<code>tool_bottleneck</code>	Which production stage is the constraint right now?
<code>tool_plan</code>	Given shared capacity, what should we run and in what order?
<code>tool_overview</code>	The factory at a glance
<code>search_workflow_docs</code>	RAG over the free-text workflow document

Production rate comes from `ProductionEvent` rows at the final stage (`FF`), averaged over working days in a 14-day lookback. Where a style has no history, a configurable assumed capacity keeps the engine answering "what rate would you need?" rather than refusing.

RAG is used for **exactly one thing** — the free-text workflow document. Everything numeric goes through a tool.

12.5 The two endpoints

- `POST /chat` — the full JSON answer.
- `POST /chat/stream` — the same answer, streamed word-by-word over SSE for a typing effect. The final frame carries `{done: true, tool, data}`.

A note for whoever wires a real model later: the deterministic backend computes the whole answer up front, so `/stream` currently chunks a finished string. When a real token stream is wired in, swap the chunker — **the endpoint shape and the frontend do not change.**

PART C — FOR FRONTEND DEVELOPERS

Everything here can be built against the mock payloads in the companion *API Reference*. You do not need a running backend.

13. The screen inventory

13.1 Stage 1 — Intake

Screen: Submission workspace

Submission #a3f2...		status: OPEN	
ORDER SHEET		SPEC SHEET	
drop file here		drop file here	
☒ accepted · BOGGI		☐ missing	
order-boggi-ss27.xlsx			
⚠ Blocking: spec_sheet missing			
[Generate breakdown] ← disabled until ready_for_stage_2			

Build:

- Two independent dropzones with their own upload state.
- A **rejection panel** — this is the highest-value screen in Stage 1. Render `signals_expected` vs `signals_found` side by side, the `reason_code`, the `closest_client_profile`, and `suggested_fix` prominently.
- A **Force accept** action shown **only** when `reason_code == "needs_manual_review"`. Re-post with `?force=true`. Never show it for a virus, size, MIME or corrupt-file rejection.
- The `blocking` array renders directly as a checklist.

13.2 Stage 2 — Breakdown & style selection

Screen: Style breakdown

Breakdown · 3 styles		✓ ready	
CLERMONT	qty 60	46:10 48:20 50:20 52:10	
Colours: BLACK 40 · COGNAC 20			
Spec	[CLERMONT_SPEC.pdf]	● suggested	[Confirm] [Change] [X]
DXF	[CLERMONT_P53.dxf]	● suggested	[Confirm] [Change] [X]
[Generate BOM] ← disabled			
CARNABY	qty 40	...	
Spec	□ none – upload one	DXF ✓ confirmed	
[Generate BOM] ← enabled			

Build:

- Three status colours for the match chips: `none` (grey), `suggested` (amber), `confirmed` (green).
- Disable "Generate BOM" whenever `spec_match_status == "suggested"`**. The backend returns `422`; do not make the user discover that.
- `none` is fine — generation proceeds with a `spec_pending` warning. Say so in the tooltip.
- Show `warnings[]` per style and per colour as an expandable list.
- Colours are a **display dimension**, not separate BOMs. One BOM per style; colours ride inside it.

13.3 Stage 2/3 — The BOM editor

Screen: BOM editor

BOM · CLERMONT		status: DRAFT		rev 3	FOB \$107.25	
CAT	MATERIAL	DCM	UOM	RATE	BULK	SOURCE
MAIN	Sheep Glass	34.50	dm²	1.80	2070.0	● 0.95
SUB	Goat Suede	2.60	dm²	2.10	156.0	● 0.50 ← AI estimate
LINE	Viscose Lining	1.20	mtr	3.40	72.0	● 0.70
MFG	Cutting+Stitch	1	–	30.00	–	–
bulk total \$6,435.00		[Confirm Cutting]				

Build:

- `dcm_source` is the single most important cell. Colour it by confidence: `template` / `manual` green, `dx` teal, `similar_style` amber, `ai_estimate` / `provisional` red with a warning icon. The cutting manager's whole job is to find the red rows and fix them.
- Batch every edit.** Collect changes, send one PATCH with `base_revision`.
- On `409 stale_revision` → toast "someone else edited this", reload, re-present the user's pending edits against the fresh tree.
- On `reconfirm_required: true` → a prominent banner: "You changed a quantity. The cutting confirmation has been revoked and this BOM is back to draft."

- The response's `recomputed` is the full new tree — render it, do not recompute client-side.
- Editable fields are only `dcm`, `qty_per_garment`, `unit_price`. Everything else is read-only.

13.4 Stage 3 — MD approval

- **Disable Approve when** `cutting_confirmed_at` is null, with the reason on hover.
- Reject requires a non-empty reason — validate before sending.
- The approve response carries `inventory_check_id` → route straight to the check result. **This is the natural hand-off point in the UI.**
- Show Reopen only on `rejected`. Warn that it clears the cutting confirmation.

13.5 Notifications

- A bell with an unread count from `GET /notifications?unread_only=true`.
- Open an `EventSource` on `/notifications/stream` for live push; fall back to polling if it drops.
- **Opening a notification must POST** `/notifications/{id}/open` — that is what cancels the email escalation. Do not skip it.
- `entity_type` + `entity_id` give you the deep link.

13.6 Stage 4 — Inventory

Screen: Stock sync — a two-step preview → commit. Always render the preview (`kept`, `dropped`, `warnings`, sample rows) and require an explicit confirm. **Warn that keys absent from the sheet will be deactivated.**

Screen: Check result

Inventory Check · CLERMONT				● OUT OF STOCK	shortfall ₹1.4L	
12 lines · 7 sufficient · 3 partial · 2 out of stock						
MATERIAL	REQUIRED	ON HAND	AVAIL.	SHORTFALL	STATUS	
Sheep Glass	2070.0	3400.0	2400.0	0.0	● sufficient	
Goat Suede	156.0	100.0	100.0	56.0	● partial	
Viscose Lining	72.0	0.0	0.0	72.0	● out_of_st.	
YKK Zip #5	60.0	—	—	60.0	● unmatched	

① Excluded (services): Cutting+Stitching · FOB charge

Build:

- Badge from `summary.badge`; the row status from `line.status`.
- **Flags are as important as the status.** `unmatched` means we could not find it in stock at all — that is a data problem, not a stock problem. `uom_mismatch` means the units did not reconcile. Render both as distinct chips.
- When `suggestion` is present, show it with a "Confirm this match" affordance.
- Always render the `excluded` list, greyed, with an explanation — otherwise the user wonders why manufacturing is missing.
- `available_qty` is often below `on_hand_qty`. Add a tooltip: "reserved by other orders".

Screen: Inventory dashboard — a client → order → style tree with a badge per style. Filterable by client or order.

13.7 Stage 5 — Suppliers and POs

Screen: PO list — filter by `status` , `needs_supplier` , `bom_id` , `supplier_id` . Give `needs_supplier: true` its own prominent filter tab — those are the POs blocking progress.

Screen: PO detail — four regions:

- 1. **Header** — number (or "not yet allocated"), status, supplier, buyer ref, dates, terms.
- 2. **Lines** — editable while pre-send.
- 3. **Money** — subtotal, then CGST+SGST or IGST depending on `gst_mode` , round-off, total. **Never render both.**
- 4. **Tracking timeline** — sent → opened → clicked → escalation rung → acknowledged.

The supplier-selection panel for `needs_supplier` POs is the highest-value screen in Stage 5. Render `candidates.ranked` as a comparison table — score, transaction count, last rate, last purchase date, contactable — and let the buyer pick. Show `candidates.ambiguous` as an explicit "too close to call — please choose" banner. Include a "none of these — add a new supplier" path into supplier creation.

Status-driven action buttons:

Status	Show
<code>draft</code>	Edit · Submit · Cancel
<code>pending_approval</code>	Approve · Reject (<i>only if the user's role matches the routing</i>)
<code>approved</code>	Send · Cancel
<code>rejected</code>	Edit · Resubmit
<code>sent</code> / <code>escalated</code>	Acknowledge manually · view tracking. No edit
<code>confirmed</code> / <code>cancelled</code>	Read-only

The escalation widget:

sent whatsapp call exhausted

✓

 14:02 ⌚ in 3h

Drive it from `current_rung` and `next_escalation_at` . Show a live countdown. When `no_contact_channel` is true, replace the whole widget with a red "No contact channel — this supplier must be reached manually."

Screen: Production board — a kanban across the nine rungs, or a table with a progress bar. Manual transitions only for MD/DM/Cutting, and only forward in the normal case.

13.8 The chatbot

A message list with `POST /chat/stream` via SSE. Each frame is `{delta}` ; the terminal frame is `{done, tool, data}` . Render `data` as a structured card beneath the prose — that is the exact number, and it is worth showing.

14. State machines you must render

BOM

From	Action	To	Guard
draft	confirm-cutting	ready_for_review	Cutting Mgr / MD / DM
ready_for_review	approve	approved locked	MD; requires cutting_confirmed_at
ready_for_review	reject	rejected	MD; reason required
rejected	reopen	draft	DM/MD; bumps revision, clears confirmation
approved / locked	export	exported	MD
draft / ready_for_review	edit a quantity	draft	Revokes confirmation if it was set

Purchase order

From	Action	To	Guard
draft rejected	submit	pending_approval	Supplier assigned; ≥ 1 line
pending_approval	approve	approved	Routed role
pending_approval	reject	rejected	Routed role; reason required
approved	send	sent	Allocates number, renders PDF
sent	(supplier engages)	responded	System
sent / responded	(clock expires)	escalated	Sweeper
any	acknowledge	confirmed	Any channel
any pre-send	cancel	cancelled	

Production board — forward-only along the nine rungs; released_to_production and beyond are manual.

15. Async patterns: 202-and-poll, SSE, and long jobs

15.1 The 202-and-poll pattern

Two operations are too slow for a request: **breakdown extraction** and **BOM generation**. Both return 202 immediately.

```
POST .../order-breakdown
|
├─ 202 {status: "queued", task_id}           → start polling
├─ 202 {status: "already_processing"}        → start polling (someone else started it)
├─ 202 {status: "already_ready"}             → fetch it now, no polling
└─ 409                                       → not eligible; show why

↓

GET .../order-breakdown  every 3-5s
├─ {status: "not_started"}  → keep polling briefly, then offer a retry
├─ {status: "processing"}   → keep polling; show a spinner with elapsed time
└─ {status: "ready", styles} → render, stop polling
```

Advice: extraction takes 30–200 seconds. Poll every 3–5 seconds, show elapsed time, and never poll forever — after ~5 minutes, offer a manual refresh rather than spinning indefinitely.

For BOM generation, the same shape: poll the style row until `bom_id` is non-null.

15.2 SSE

Two streams: `/procurement/notifications/stream` and `/chat/stream`.

```
const es = new EventSource(url, { withCredentials: true });
es.onmessage = (e) => { /* JSON.parse(e.data) */ };
es.onerror = () => { es.close(); /* fall back to polling */ };
```

Frames starting with `:` are keep-alive comments — `EventSource` ignores them for you. Always keep a polling fallback; the notification table is the source of truth, so nothing is lost.

15.3 File uploads

`multipart/form-data`, field name `file`. Cap client-side at `MAX_UPLOAD_MB` (default 25) to save the user the round trip — but be aware the server aborts mid-stream at the cap and returns `413`.

16. The error contract

16.1 Two shapes

Stage-1 upload rejections are a rich flat envelope:

```
{
  "error": "document_validation_failed",
  "reason_code": "not_an_order_sheet",
  "submission_id": "...",
  "document_fingerprint": { "filename": "...", "mime": "...", "sha256": "...", "size_bytes": 84213 },
  "validation": {
    "status": "rejected",
    "reason_code": "not_an_order_sheet",
    "expected_kind": "order_sheet",
    "confidence": 0.31,
    "signals_expected": ["ORDER CONFIRMATION", "TAGLIA", "QTY"],
    "signals_found": ["PACKING LIST", "CARTON"],
    "closest_client_profile": "BOGGI",
    "method": "llm",
    "suggested_fix": "Upload the order confirmation, not the packing list."
  }
}
```

Everything else is FastAPI's `detail` wrapper:

```
{ "detail": { "error": "stale_revision", "current_revision": 7 } }
```

Some errors are a plain string: `{"detail": "BOM not found."}`. **Handle both.**

```
function errorCode(body) {
  if (body?.error) return body.error;           // stage-1 envelope
  const d = body?.detail;
  if (typeof d === "string") return d;          // plain message
  return d?.error ?? "unknown_error";          // structured detail
}
```

16.2 Status codes

Code	Means	Do
400	Malformed tracking link	—
401	Missing/expired token	Redirect to login
403	Wrong role	Show who <i>can</i> do it — the body often names the roles
404	Not found	—
409	State conflict — the interesting one	See below
413	File too large	Show the cap
415	Unsupported file type	Show accepted types
422	Validation failed	Field-level or document-level
500	Server error	Show the <code>request_id</code> — support can trace it
503	Virus scanner unavailable	Offer retry

16.3 The 409s worth handling individually

error	Meaning	UI response
stale_revision	Someone else edited	Reload; re-present the user's edits
bom_locked / po_locked	Frozen	Disable editing; explain
bom_rejected	Must reopen first	Offer the Reopen action
cutting_confirmation_required	MD tried to approve too early	Disable Approve upfront
spec_suggestion_unconfirmed	Spec still suggested	Disable Generate BOM upfront
bom_not_approved	Check run on an unapproved BOM	Disable
no_inventory_check	POs generated before the check	Offer "run the check first"
needs_supplier	Submit without a supplier	Open the supplier picker
empty_po	No lines	Disable Submit
duplicate_content	Same bytes already on record	Explain where; the body says
wrong_approver	Not the routed role	Name the roles from the body
not_pending_approval, not_submittable, not_approved, not_rejected, not_ready_for_review	State-machine violations	Drive buttons from status; these should not happen

The pattern: almost every 409 here is avoidable by driving button state off the status the API already gave you.

17. Badges, chips and status vocabulary

Consistent colour across the whole app:

Family	Value	Colour	Icon
Document	accepted	green	✓
	pending	grey	...
	needs_manual_review	amber	⚠
	rejected	red	✗
	superseded	grey, struck	↶
Scan	clean / skipped	green / grey	🛡
	infected / error	red	☣
Match	confirmed	green	✓
	suggested	amber — needs a click	?
	none	grey outline	👉
BOM	draft	grey	✎
	ready_for_review	blue	👁
	approved / locked	green	✓ / 🔒
	rejected	red	✗
	exported	teal	📄
DCM source	manual (1.00)	green	👉
	template (0.95)	green	💾
	dxg (0.88)	teal	📐
	similar_style (0.70)	amber	≈
	ai_estimate (0.50)	red	🖨
	provisional	red	⚠
Stock line	sufficient	green	✓
	partial	amber	◐
	out_of_stock	red	✗
Stock flag	unmatched	red outline	?
	uom_mismatch	amber outline	↔
	suggestion	blue outline	💡
PO	draft grey · pending_approval blue · approved green		
	sent teal · responded teal · escalated amber · confirmed green · rejected / cancelled red		
Board	Nine rungs — a gradient grey → blue → green works well		

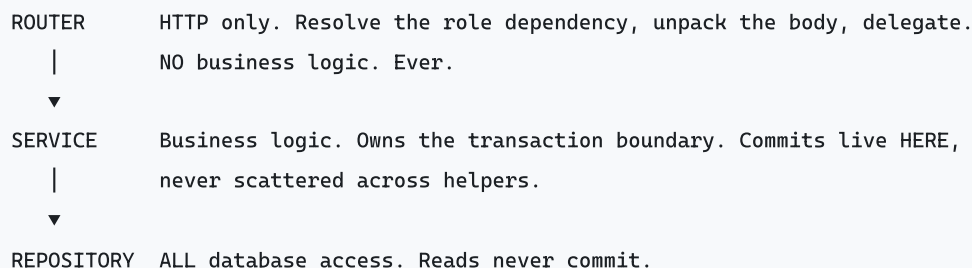
PART D — FOR BACKEND DEVELOPERS

18. Architecture and the layering rules

18.1 A modular monolith

One deployable FastAPI app; many internal modules. Cross-module calls go **through services**, never repositories or models — so a module can later be lifted out without a rewrite.

18.2 The three layers



Two supporting layers:

- `presenters.py` — pure ORM-row → JSON serialisation. No DB, no rules. Keeps services focused on orchestration.
- `enums.py` / `schemas.py` — value sets and Pydantic request bodies.

18.3 The cross-module rules

1. **Service → service only.** `InventoryService` reads a BOM via `BomService.get_bom_dto()`, never `BomRepository`.
2. **DTOs at the boundary.** Cross-module reads return `SimpleNamespace` DTOs, not ORM rows — no lazy-loading surprises across a module line.
3. **Foreign keys are table-name strings.** `ForeignKey("bom.id")`, never a Python import of the owning module's model.
4. **Relationships stay inside a module.**
5. **Lazy imports inside methods** keep the module graph acyclic. `bom.service → inventory.service`, `inventory.service → supplier_po.production_tracking_service`, and `supplier_po.po_service → bom.service` are all imported *inside the method that uses them*. **Do not hoist them** — this is enforced by `.importlinter`.

18.4 Async discipline

Every blocking call goes through `run_in_threadpool`: `openpyxl`, `pypdf`, `libmagic`, `clamd`, LLM calls, PDF rendering, email sending, WhatsApp/voice transport. **The event loop is never blocked.**

18.5 Portability

IDs are `uuid4()` generated in the app, not `gen_random_uuid()`. No `JSONB` operators, no `ON CONFLICT`. Status columns are `VARCHAR` storing a `str`-Enum's `.value` — **never a native Postgres enum** in these modules (unlike Phase 1's `app_user.role`). Keep it that way; a future Oracle migration depends on it.

19. Module map — every file and what it is for

19.1 `procurement/` — Stage 1 intake

File	Lines	Purpose
<code>router.py</code>	185	7 endpoints. Streams uploads with a cap, maps <code>UploadError</code> → status
<code>service.py</code>	508	The brain. Pairing, the pipeline, sha-idempotency, supersede, the gate
<code>repository.py</code>	108	All DB access, plus the atomic CAS for the breakdown claim
<code>pipeline.py</code>	146	<code>process_upload</code> — scan → sniff → validate → quarantine/promote
<code>validator.py</code>	467	Anchors, fingerprints, grid signals, thresholds, closest-profile
<code>classifier.py</code>	316	LLM + vision classifiers, injectable for tests
<code>sniffing.py</code>	381	MIME detection, text-layer extraction, page counting
<code>scanning.py</code>	126	Virus-scan abstraction (clamd + a skip driver)
<code>registry.py</code>	182	<code>ProfileView</code> — a session-free view of a <code>client_template</code>
<code>presenters.py</code>	255	The success / rejection / duplicate envelopes
<code>errors.py</code>	51	<code>UploadError</code> → stable HTTP statuses
<code>models.py</code>	108	<code>Submission</code> , <code>ClientTemplate</code>
<code>enums.py</code>	93	7 value sets including the <code>RejectReason</code> catalogue
<code>seed_templates.py</code>	75	Idempotent seeding from YAML

19.2 bom/ — Stage 2/3

File	Lines	Purpose
<code>service.py</code>	1850	The brain. Breakdown, generation, the DCM cascade, edit, confirm, approve, reject, reopen, export, admin config
<code>models.py</code>	595	20 tables — BOM, items, spec, POM, templates, patterns, staging, config
<code>extraction.py</code>	767	Document → <code>ExtractedSpec</code> / <code>ExtractedOrder</code>
<code>extraction_schemas.py</code>	487	The typed extraction contracts
<code>repository.py</code>	550	All DB access
<code>notification_service.py</code>	356	In-app notices, SSE, the 2-hour escalation sweep
<code>dxg_pattern.py</code>	331	DXF parsing — pieces, areas, fabrics, sizes
<code>router.py</code>	288	19 endpoints
<code>attribution.py</code>	253	Spec material → BOM line attribution
<code>config_store.py</code>	240	Hot-reloadable config snapshot (yields, roles, catalogue, checks)
<code>tasks.py</code>	220	5 Celery tasks
<code>seed_stage2.py</code>	205	Garment types, POM dictionary seeding
<code>export.py</code>	177	PDF rendering — WeasyPrint → ReportLab → HTML
<code>checks.py</code>	176	Per-client BOM sanity rules
<code>schemas.py</code>	152	Request bodies + breakdown response models
<code>pattern.py</code>	143	Pattern resolution and <code>learn_yield</code>
<code>pdf_content.py</code>	136	PDF text/table extraction
<code>fabric_roles.py</code>	130	Fabric label → role/category/is-leather
<code>dcm.py</code>	118	The DCM math — signature, area heuristic, species, confidence map
<code>costing.py</code>	98	Pure cost math. No DB, no I/O
<code>enums.py</code>	54	<code>BomStatus</code> , <code>BomItemCategory</code> , <code>DcmSource</code> , <code>ExtractionSource</code>
<code>excel_content.py</code>	292	Excel extraction
<code>units.py</code>	18	Price normalisation

19.3 inventory/ — Stage 4

File	Lines	Purpose
service.py	363	The brain. Sync, the check, reservations, the dashboard
presenters.py	197	Check and dashboard shapes
inventory_import.py	182	Spreadsheet → normalised rows
repository.py	181	DB access — including the <code>FOR UPDATE</code> candidate fetch
seed_inventory.py	138	Alias + UOM seeding
models.py	134	5 tables
inventory_normalize.py	131	<code>normalize_key</code> , <code>bom_line_key</code> , <code>tokens</code> — the shared normaliser
inventory_match.py	130	<code>match_line</code> , <code>convert_on_hand</code> , the <code>Alias</code> type
router.py	106	7 endpoints
enums.py	38	4 value sets

19.4 supplier_po/ — Stage 5

File	Lines	Purpose
po_service.py	842	The brain. Generate, edit, cross-check, send, track, escalate, acknowledge
supplier_service.py	291	Import, CRUD, <code>match_line</code>
router.py	284	26 endpoints including the unauthenticated tracking + webhook routes
models.py	246	7 tables
repository.py	235	DB access, PO numbering, due-escalation queries
supplier_import.py	265	Contacts + provision-ledger parsing
po_export.py	205	PO PDF rendering
supplier_match.py	206	Pure matching + ranking. No DB
production_tracking_service.py	177	The board
supplier_normalize.py	136	Name normalisation, GSTIN → state code
escalation.py	118	WhatsApp + voice transport abstraction
presenters.py	115	PO / supplier / board shapes
po_costing.py	95	Pure GST math
po_tracking.py	94	Token minting, pixel bytes, link wrapping, signature verify
schemas.py	90	Request bodies
enums.py	86	7 value sets
po_templates.py	80	Per-supplier-type PO templates

19.5 intelligence/ — the chatbot

File	Lines	Purpose
forecast.py	224	Pure forecasting math. Schedule status, bottleneck, planning
tools.py	201	DB + math tools — the exact answers
agent.py	195	The deterministic keyword router
langgraph_agent.py	185	The real ReAct agent
models_catalog.py	99	Swappable embedding + chat model picks
rag.py	107	FAISS vector search over the workflow document
router.py	57	2 endpoints
service.py	32	Thin wrapper — also callable from a scheduled job
schemas.py	13	ChatRequest / ChatResponse

20. The data model

20.1 Conventions

Every table: UUIDMixin (a portable GUID primary key) + TimestampMixin . Money is Numeric(12,2) ; fractional quantities Numeric(12,3) ; confidences Numeric(5,4) ; semi-structured data JSON_VARIANT .

20.2 Stage 1 — 2 tables (+ 3 in core)

submission	the pairing batch. Two slot pointers + a membership set
client_template	per client × doc-kind validation profile
(core) document	every uploaded file ever, sha256 globally unique
(core) notification	the notification table both BOM and PO write to
(core) audit_log	every consequential action

A schema note worth knowing: there are two FK cycles here — document ↔ submission , and the three-table document → submission → client_order → document . Both are broken with use_alter=True so Alembic and create_all can order the CREATEs. All three slot FKs are ondelete="SET NULL" — they are pointers, and losing a pointer must never cascade into losing a document.

20.3 Stage 2/3 — 20 tables

CORE

bom	header + state machine. UNIQUE (client_order_id, style_id)
bom_item	one line. qty_per_garment IS the DCM for material lines
spec_sheet	a parsed spec – typed columns + JSON behind a spec_type discriminator

BREAKDOWN (fan-out v2)

order_style	one style off the order sheet + its match state + bom_id
order_style_color	per-colour quantities under a style

MEASUREMENT

garment_type	required POMs + the Source-3 area formula + default wastage
pom_dictionary	native term → standard pom_code (seeded)
pom_measurement	one standardised POM per spec per size

THE MEMORY

style_consumption_template	* the DCM memory. Cross-order-stable key
dx_f_yield_observation	learned yield per species
dx_f_yield	the configured yield factor per species

PATTERNS

pattern_reference	"follow pattern X in size Y" → resolved style / template
pattern_extraction	a parsed DXF – area + fabric matrices
pattern_piece	one panel: block, name, fabric, size, qty, net area

STAGING (audit truth, pre-promotion)

spec_extraction	the raw ExtractedSpec + denormalised columns
order_extraction	the raw ExtractedOrder + denormalised columns

CONFIG (hot-reloadable)

fabric_role	fabric label → role/category/is_leather
cost_catalog_line	per-garment-code default cost lines
client_check_rule	per-client BOM sanity rules
material_rate	current material rates by normalised key

20.4 Stage 4 — 5 tables

inventory_item	the normalised, deduplicated stock master
inventory_check	one report per BOM (running → complete)
inventory_check_line	per-BOM-item verdict + exact shortfall + flags
inventory_reservation	* the SOFT ledger. available = on_hand - sum(active)
material_alias	curated BOM term → inventory key
uom_conversion	from_uom → to_uom → factor

20.5 Stage 5 — 7 tables

supplier	the vendor directory + send target
supplier_supply_history	* the matching index. Pre-aggregated (supplier, article) evidence
purchase_order	money + the cross-check/send/escalation state machine
po_item	one line, keeping bom_item / inventory_item back-links
po_response	one contact attempt (email / whatsapp / call) + its result
po_tracking_event	the unified engagement log
production_tracking	one row per (order, style) on the nine-rung ladder

20.6 Material vs Inventory — the two stock systems

They are not duplicates and must not be merged.

	material (Phase 1)	inventory (Phase 2)
Driven by	A human typing a lot in	A BOM plus a spreadsheet upload
Keyed to	Category / subtype / article	A bom_id
Has	Lot barcodes, three floor categories, per-type required fields	MaterialAlias , UomConversion , BOM-keyed checks and reservations
Core operations	create_lot , stock , receive , supplier orders	run_check(bom_id) , release_reservations(bom_id) , alias matching

Why not reuse **inventory for Phase 1?** Every operation is keyed to a **bom_id** , it ingests spreadsheets, and it carries alias/UOM machinery for matching messy BOM text — none of which the manual floor flow has or wants. In Phase 1 there is no BOM.

Why not overwrite **inventory with **material** ?** The BOM module *depends on* **run_inventory_check** and **release_reservations** . Replacing it breaks the pipeline before it ships.

The bridge: the physical stock the BOM's inventory check reads against **is** the material lots. The manual "manager checks stock and places an order" step is what gets replaced; the storage layer stays. Keeping them separate now is exactly what makes that a bridge rather than a rewrite.

21. Cross-module choreography — who calls whom, and when

```
ProcurementService.upload_*
  | (accepted, both slots filled)
  v
BomService.build_order_breakdown          [Celery: bom.build_order_breakdown_for_submission]
  | <- ProcurementService.spec_documents_for_client      (candidate specs)
  | <- ProcurementService.claim_submission_for_breakdown (atomic CAS)
  v
BomService.confirm_style_attachments       [synchronous – two column writes]
  v
BomService.generate_bom_for_style          [Celery: bom.generate_bom_for_style]
  \--> BomService.generate_bom
  v
BomService.confirm_cutting
  |--> repo.upsert_consumption_template * THE LEARNING STEP
  |--> repo.add_yield_observation
  \--> NotificationService.create_review_notifications
  v
BomService.approve_bom                    <- MD only
  |--> ClientService.create_order_with_breakdown * the order becomes real
  |--> ProcurementService.link_submission_to_order
  |--> ProductionTrackingService.on_bom_approved (best-effort)
  \--> InventoryService.run_check                (best-effort)
        |--> BomService.get_bom_dto              (strict DTO boundary)
        \--> ProductionTrackingService.on_inventory_checked
  v
PoService.generate_for_bom
  |--> BomService.get_bom_dto
  |--> InventoryService.get_latest_check_dto        (the shortfall lines)
  |--> InventoryService.get_alias_pairs
  \--> SupplierService.match_line                  (per shortfall line)
  v
PoService.send_po
  \--> ProductionTrackingService.on_po_raised
  v
PoService._acknowledge                     <- any channel
  \--> ProductionTrackingService.on_po_confirmed -> MATERIAL_READY when all confirmed
  v
ProductionTrackingService.transition        <- HUMAN: released_to_production
  v
===== PHASE 1 TAKES OVER =====
```

Three properties worth internalising:

1. **Every cross-module read goes through the owning service and returns a DTO.** `InventoryService` never imports `BomRepository`. `PoService` never imports inventory models.
2. **Every side-effect edge is best-effort.** The board and the inventory check are wrapped so a failure there cannot fail the approval. Progress tracking must never block progress.
3. **Every import in this diagram is lazy** — inside the method. Hoisting one creates a cycle.

22. The key algorithms

22.1 The DCM cascade — `bom/dcm.py` + `BomService._resolve_dcm`

```
CONFIDENCE = {  
    TEMPLATE:      0.95,    # the memory – cheapest and best  
    DXF:           0.88,    # the CAD pattern  
    SIMILAR_STYLE: 0.70,  
    AI_ESTIMATE:   0.50,    # last resort – FLAGGED  
    MANUAL:        1.00,    # a human said so  
}
```

Resolution order is strict; **a lower-numbered source always wins where available**, and every material line is stamped with its source and confidence.

The Source-3 heuristic is the only place finished measurements ever touch consumption:

```
length_cm = sum(weight * POM) over the garment type's length POMs  
width_cm  = sum(weight * POM) over its width POMs  
dcm       = panels * length * width * calibration / 100 * (1 + wastage)
```

It returns `None` when the required POMs are absent — a garment type with no POMs *can never* use Source 3, by design. That is why `provisional` exists: no source resolved, quantity 0, flagged, awaiting a human.

The style signature — `customer_ref` → `internal_ref` → `slug(name)`, uppercased — is what makes the memory work across orders. It deliberately avoids `style_id` because style rows are order-scoped.

22.2 Inventory matching — `inventory/inventory_match.py`

`key` → `alias` → *fuzzy suggestion only* → unmatched. A fuzzy candidate above the Jaccard floor is **surfaced, never bound**. Confirming one writes a `material_alias` row, so it becomes a clean `alias` match next time.

22.3 Supplier ranking — `supplier_po/supplier_match.py`

```
score = 1.0 * recency          (exponential decay, ~half per year)  
       + 0.6 * frequency  
       + 0.8 * contactable  
       - 0.3 * normalised_rate
```

Held as `ambiguous` when the hit is category-only (weak) or the top two fall within a 0.12 tie band. **Contactability is a ranking term, not a filter** — a frequent contactless vendor still surfaces; their PO is flagged `no_contact_channel` downstream.

22.4 GST — `supplier_po/po_costing.py`

```
amount      = qty * unit_price          per line
subtotal    = sum(amount)
if supplier_state == buyer_state: cgst = sgst = subtotal * rate/2 ; igst = 0  -> INTRA
else:                                igst = subtotal * rate          ; cgst=sgst=0 -> INTER
total       = subtotal + taxes + round_off
```

22.5 PO numbering

Financial-year scoped, April to March: `2025-04-15` → `25-26` ; `2025-03-15` → `24-25` . Allocated **at send**, not at generation — a draft that never ships never burns a number.

22.6 Optimistic concurrency

Both the BOM and the PO use identical machinery:

- 1. Client sends `base_revision` + a bundle of edits
- 2. Service refuses on a state mismatch (locked / rejected / sent)
- 3. Service refuses on a revision mismatch (409 with the current revision)
- 4. Service applies edits and recomputes
- 5. Service ATOMICALLY claims the revision: `UPDATE ... WHERE revision = base`
- 6. Zero rows updated -> rollback, re-read, 409 with the fresh revision

Step 5 is what makes it correct. The check at step 3 is a courtesy; the claim at step 5 is the guarantee.

22.7 Idempotency, everywhere

Operation	Mechanism
Document upload	sha256 cache — replays the stored verdict, no second AI bill
Breakdown	Existing rows → <code>already_ready</code> ; claim held → <code>already_processing</code>
BOM generation	<code>bom_id</code> already set → replay
PO generation	Non-cancelled POs exist → <code>already_generated</code>
Inventory sync	Upsert on <code>normalized_key</code>
Inventory check	Releases prior reservations first, then re-claims
BOM / PO export	sha256 document dedupe
Acknowledgement	<code>acknowledged_at</code> non-null → no-op
Escalation sweep	<code>acknowledged_at IS NULL</code> + <code>current_rung < 3</code> guards
Notification escalation	<code>NOT EXISTS</code> guard on the child email

22.8 Transaction boundaries

Operation	Boundary
BOM generation	ONE — staged with <code>add</code> / <code>flush</code> , committed once at the end
BOM edit	One, with the revision claim inside it
Inventory check	ONE , with candidates locked <code>FOR UPDATE</code>
PO generation	One
Escalation sweep	One per sweep across all due POs

The BOM generation rule, stated plainly: the repository read helpers used in the generate path **must not commit**. Reads never should. If one does, the single-commit guarantee silently breaks and a mid-generation failure leaves half-written rows.

23. Background work

23.1 Celery tasks

Task	Purpose	Why not inline
<code>bom.build_order_breakdown_for_submission</code>	Extract the order document → <code>OrderStyle</code> rows	30–200s
<code>bom.generate_bom_for_style</code>	Generate one BOM	30–200s
<code>app.modules.bom.tasks.parse_pattern_dxf</code>	Parse an uploaded DXF	CPU-bound
<code>app.modules.bom.tasks.scan_freight_risk</code>	Periodic freight-risk scan	Scheduled
<code>app.modules.bom.tasks.escalate_review_notifications</code>	Celery-beat alternative to the in-process sweeper	Scheduled

Files are handed to workers as a **storage key**, never as bytes on the queue.

23.2 In-process sweepers

Both are started in the single `lifespan` and cancelled cleanly on shutdown.

Sweeper	Interval	Does
<code>_notification_sweeper</code>	<code>notification_sweep_seconds</code> (60)	Emails BOM-review notices unopened after 2 hours
<code>_po_escalation_sweeper</code>	Same	Advances one escalation rung per due PO

The single-replica caveat. Both sweepers are DB-driven and idempotent, so a second replica cannot double-send — but it *will* do duplicate work. In a multi-replica deployment, disable them (`notification_sweeper_enabled=false` , `po_escalation_sweeper_enabled=false`) and run the Celery-beat equivalents instead.

`lifespan` **must be defined exactly once**. A second definition silently overrides the first — this bug once stopped both sweepers from ever starting.

24. Configuration and migration notes

24.1 Settings that change behaviour

Setting	Default	Effect
<code>max_upload_mb</code>	25	Upload cap; aborts mid-stream
<code>bom_review_escalation_hours</code>	2	Internal BOM-review nudge
<code>notification_sweeper_enabled</code>	true	Disable on multi-replica
<code>notification_sweep_seconds</code>	60	Both sweepers' interval
<code>frontend_base_url</code>	<code>http://localhost:3000</code>	Deep links in notifications and emails
<code>po_buyer_state_code</code>	<code>33</code>	Drives intra/inter GST
<code>po_gst_rate</code>	12.0	6+6 intra, 12 inter
<code>po_default_delivery_days</code>	10	
<code>po_default_payment_terms_days</code>	60	
<code>po_escalation_hours</code>	5	External supplier window — distinct from the 2h internal one
<code>po_escalation_sweeper_enabled</code>	true	Disable on multi-replica
<code>po_tracking_base_url</code>	—	Must be publicly reachable — supplier mail clients call it
<code>escalation_transport</code>	<code>log</code>	<code>log</code> <code>noop</code> <code>twilio</code>
<code>chat_model</code>	unset	Unset → deterministic router; set → LangGraph agent

24.2 Seeded configuration

Source	Seeds
<code>config/client_templates.yaml</code>	Per-client validation profiles
<code>config/material_aliases.yaml</code>	BOM term → inventory key
<code>config/uom_conversions.yaml</code>	Unit factors (identity rows included)
<code>config/po_templates.yaml</code>	Per-supplier-type PO layouts
<code>bom/seed_stage2.py</code>	Garment types, POM dictionary

All seeding is **idempotent**. Several config sets are additionally hot-editable through admin endpoints, which refresh the in-memory `config_store` snapshot without a restart.

24.3 Migration notes

- **No native Postgres enums in these modules.** Every status is `VARCHAR` storing a `str`-Enum's `.value`. Adding a value is a Python change only. (*This is the opposite of Phase 1's `app_user.role`, which is a native `user_role` enum and needs `ALTER TYPE ... ADD VALUE`.*)
- **Every model module must be imported in `main.py`** so `Base.metadata` sees every table. A missed import makes Alembic autogenerate try to **DROP** the table — the schema-drift trap.
- Set `down_revision` to the current head (`alembic heads`) before running.
- IDs come from the app (`uuid4`), not `gen_random_uuid()`. Keep it that way for portability.

24.4 Known traps — do not reintroduce

Trap	Symptom
Importing <code>api</code> from <code>sqlalchemy.event</code>	Wrong object; the router never registers. Use <code>app.include_router</code>
Defining <code>lifespan</code> twice	The second silently overrides — sweepers never start
Hoisting a lazy cross-module import	Import cycle
A repository read that commits	Breaks the single-transaction guarantee in BOM generation
Adding a model without importing it in <code>main.py</code>	Alembic tries to drop the table

25. A code-review checklist

Use this when reviewing a change to any of the five modules.

Layering

- [] Router contains **no** business logic — resolves the dependency, unpacks, delegates.
- [] Commits live in the service, not scattered through helpers.
- [] All DB access is in the repository.
- [] Cross-module reads go through the owning **service** and return a **DTO**.
- [] Cross-module FKs are table-name strings.
- [] Cross-module imports are **lazy** (inside the method).

Correctness

- [] Every blocking call is in `run_in_threadpool`.
- [] Money and quantities are `Decimal` throughout — no float drift.
- [] Derived values (`available` , `total_cost` , `bulk_total`) are **computed at write time**, never on read, never trusted from the client.
- [] Mutating endpoints are idempotent, or explicitly documented as not.
- [] Concurrent mutations are guarded — a revision claim, a CAS, or `FOR UPDATE`.

State and safety

- [] Every state transition validates the current state and returns a structured `409`.
- [] Consequential actions write an `audit_log` row.
- [] Approval gates are hard state transitions, never soft booleans.
- [] Side-effect edges (board, check) are best-effort and cannot fail the primary action.
- [] Nothing is deleted where a soft-deactivate would do.
- [] **When the system is unsure, it asks** — no silent auto-selection of a spec, a supplier or a match.

Contract

- [] New enum values are `str`-Enum in `VARCHAR` — not a native PG enum.
- [] New models are imported in `main.py`.
- [] New endpoints carry a `require_roles` gate — or are deliberately, documentedly open.
- [] Response shapes are built in `presenters.py`, not inline in the service.

- [] Error bodies use the structured `{"error": ...}` form, so the frontend can branch on a code.
-

APPENDICES

Appendix A — Glossary

Term	Meaning
BOM	Bill of Materials — every material, quantity and cost for one style
BMO-1	The reference BOM the costing logic is grounded in
DCM	Square decimetres of a material consumed per garment. The central number in Stage 2
POM	Point of Measure — a standardised garment measurement (CHEST, SHOULDER...)
DXF	The CAD cutting-pattern file format
Style signature	The cross-order-stable key the consumption memory is stored under
Consumption template	A remembered, cutting-manager-confirmed DCM for a style
Submission	The upload batch pairing one order sheet with one spec sheet
Slot	A pointer to the current order or spec document on a submission
Supersede	Replacing a slot's document with a corrected upload; the old one is kept
Shortfall	Required quantity minus what could be reserved — what Stage 5 must buy
Reservation	A soft claim on stock. Never mutates on-hand
Alias	A curated synonym: a BOM term that means a particular inventory key
Supply history	The pre-aggregated (supplier, article) purchase ledger driving supplier matching
Cross-check	The PO approval step, routed by material type
Escalation ladder	Email → WhatsApp → call → give up
Rung	The current step on that ladder (0–3)
Tracking token	The opaque per-send identifier in the pixel, links and webhooks
Production board	The nine-rung ladder, one row per (order, style)
FOB	Free On Board — the per-garment price the BOM rolls up to
GSTIN	Indian tax registration number; its first two digits are the state code

Appendix B — Complete enum reference

SubmissionStatus	open · complete · consumed · rejected · queued
ValidationStatus	pending · accepted · rejected · superseded · needs_manual_review
ScanStatus	clean · skipped · infected · error
ClassificationMethod	heuristic · llm · manual
ExpectedLayout	scanned_pdf · digital_pdf · spreadsheet
SizeSystem	letter · eu · mixed
RejectReason	unsupported_mime (415) · file_too_large (413) · empty_or_corrupt (422) · virus_detected (422) · scanner_unavailable (503) · not_an_order_sheet (422) · not_a_spec_sheet (422) · wrong_slot (422) · needs_manual_review (422) · submission_locked (409) · duplicate_content (409)

BomStatus draft · ready_for_review · approved · rejected · locked · exported
BomItemCategory main_material · sub_material · lining · interlining · thread · accessory · manufacturing · packaging · fob_charge
DcmSource template (0.95) · dxf (0.88) · similar_style (0.70) · ai_estimate (0.50) · provisional · manual (1.00)
ExtractionSource gemini · groq · manual
InventoryCheckStatus running · complete
InventoryLineStatus sufficient · partial · out_of_stock
ReservationStatus active · released · consumed
MatchMethod key · alias · manual
POStatus draft · pending_approval · approved · rejected · sent · responded · confirmed · escalated · cancelled
POResponseChannel email · whatsapp · call
POResponseStatus pending · opened · confirmed · no_response · failed
POTrackingEventType open · click · delivery · bounce · complaint · whatsapp · call
SupplierEmailStatus unknown · valid · invalid
SupplierType leather · accessory · service
ProductionTrackingStatus awaiting_bom · bom_approved · inventory_checked · po_raised · po_confirmed · material_ready · released_to_production · in_production · completed
Match statuses (order_style.spec_match_status / dxf_match_status) none · suggested · confirmed

Appendix C — Audit log actions

Every one of these writes an `audit_log` row with the actor, entity, and a before/after snapshot.

Action	Written by
VIRUS_DETECTED	Stage-1 upload
BOM_EDIT	edit_bom_items
BOM_SUBMIT_FOR_REVIEW	confirm_cutting
BOM_APPROVE	approve_bom
BOM_BREAKDOWN	_materialize_breakdown
BOM_REJECT	reject_bom
BOM_REOPEN	reopen_bom
BOM_EXPORT	export_bom
INVENTORY_CHECK_RUN	run_check
SUPPLIER_IMPORT	Supplier commit
PO_GENERATE	generate_for_bom
PO_EDIT	edit_po
PO_SUBMIT_FOR_APPROVAL	submit_po
PO_APPROVE / PO_REJECT / PO_CANCEL	The cross-check machine
PO_SEND	send_po
PO_ACKNOWLEDGE	_acknowledge
PRODUCTION_TRACKING_UPDATE	Every board transition

KairoX ERP — Phase 2 System & Business Guide. Companion document: **KairoX Phase-2 API Reference**.